



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECR3253-02 NETWORK PROGRAMMING

Personal Reflection

GROUP MEMBERS	MATRIC NUMBER
MOHAMED OMAR MAKHLOUF	A23CS4014

Group Assignment — Network Automation Project

Group: Nahj

1. My Contribution to the Project

As the group leader of Nahj, my role went beyond just writing code, I was responsible for setting up the entire foundation that the rest of the team built on top of.

My first responsibility was designing the project architecture and deciding on the toolstack. After reviewing the assignment requirements, I proposed using Ansible as the main automation engine, Docker to containerize the Ansible control node so every team member runs the same environment, and GNS3 with a Cisco CSR1000v virtual router as the network device target. I also introduced two additions that went beyond the base requirements: Ansible Vault for encrypting credentials, and a GitHub Actions CI pipeline that runs ansible-lint on every Pull Request. These decisions were made to give the project a professional, DevSecOps-oriented quality that most groups would not have.

On the technical side, I handled all the infrastructure setup. I wrote the Dockerfile and docker-compose.yml to containerize the Ansible control node, configured the inventory file to define both the Cisco router and the Linux host as automation targets, set up the Ansible Vault encrypted credentials file, and wrote the master site.yml playbook that ties all individual playbooks together. I also configured the CSR1000v virtual router in VMware Workstation enabling SSH, setting the hostname, and verifying connectivity before handing off the router target to the team.

I also managed the GitHub repository throughout the project. I created the repo, added all members as collaborators, enforced the branch-based workflow, and reviewed Pull Requests before merging into main. I wrote the full README with the project banner, architecture diagram, and network topology, and maintained the project documentation in the doc/ folder including the overview and phases tracker.

2. Challenges Encountered During Collaboration

The most significant challenge I faced was the environment setup. My first attempt to run the GNS3 VM inside VMware Workstation failed because Windows Defender's Controlled Folder Access was blocking VMware's ovftool.exe from accessing the Virtual Machines folder. This took considerable time to diagnose since the error message was vague. After resolving that, I discovered the CSR1000v's SSH service only offered legacy key exchange algorithms (diffie-hellman-group14-sha1) that modern OpenSSH clients reject by default, which caused connection failures. I had to configure specific SSH arguments in the Ansible inventory to allow paramiko to negotiate with the router successfully.

Another challenge was the Ansible Vault password file. Since the project runs inside Docker with the project directory mounted from Windows, the `chmod` command had no effect on the file permissions due to NTFS not supporting Unix permission bits properly. Ansible was treating the vault password file as an executable script and failing. The workaround was to write the vault password to `/tmp` inside the container where permissions work correctly.

From a collaboration standpoint, coordinating five people with different schedules and technical backgrounds was harder than the technical work itself. I found that giving each member a clearly scoped file with step-by-step instructions rather than leaving them to figure out Ansible on their own was the most effective approach to keep everyone moving.

3. What I Learned from the Project

This project gave me hands-on experience applying infrastructure-as-code principles to network device management, which is something I had done previously with application deployments but not with network operating systems. Learning how Ansible's `network_cli` connection plugin works, how `cisco.ios` modules translate YAML definitions into IOS commands, and how to handle the quirks of connecting to legacy Cisco SSH implementations was genuinely valuable.

Working with Ansible Vault taught me a practical approach to secrets management in automation pipelines keeping credentials encrypted in the repository while making them accessible to automation without exposing them in plain text. This is a pattern directly applicable to real DevSecOps workflows.

The CI pipeline experience reinforced something I already believed: automated quality checks catch problems early and force cleaner code. Even for a small group assignment, having `ansible-lint` run on every PR made the team think twice before pushing untested YAML.

Most importantly, leading this project taught me that good engineering is as much about planning and communication as it is about technical execution. The decisions made in the first few days the toolstack, the repo structure, the branch workflow determined how smoothly everything else went.